

TLS - Transport Layer Security

Eine Betrachtung von TLS hinsichtlich Sicherheit und Robustheit

1st Hannah Herberstein
HKA

Hochschule Karlsruhe
heha1015@h-ka.de

2nd Leonie Leber
HKA

Hochschule Karlsruhe
lele1018@h-ka.de

3rd Sina Frieß
HKA

Hochschule Karlsruhe
frsi1018@h-ka.de

4th Niels Reisch
HKA

Hochschule Karlsruhe
reni1030@h-ka.de

Zusammenfassung—Transport Layer Security (TLS) bildet das kryptographische Fundament sicherer Netzwerkcommunication. Diese Arbeit analysiert den TLS-Handshake-Prozess, die Zertifikatsvalidierung sowie Vertrauensketten anhand zweier kontrastierender Client-Server-Szenarien: einer gehärteten TLS 1.3-Konfiguration mit Perfect Forward Secrecy und einem System mit veralteter Protokollversion und schwachen Cipher Suites. Am Beispiel des POODLE-Angriffs (CVE-2014-3566) wird ein gezielter Protokoll-Downgrade auf SSL 3.0 simuliert und anhand synthetischer Handshake-Logs nachvollzogen, an welcher Stelle die Sicherheitsmechanismen versagen. Ergänzend wird das Szenario eines selbstsignierten Zertifikats untersucht, das den Handshake mit einem CERTIFICATE_UNKNOWN-Alert abbricht und die Verwundbarkeit gegenüber Impersonation-Angriffen aufzeigt. Die Bewertung der Auswirkungen auf Vertraulichkeit, Integrität und Authentizität zeigt, dass Legacy-Protokolle alle drei Schutzziele gleichermaßen gefährden. Als Härungsmaßnahmen werden die ausschließliche Verwendung von TLS 1.2 und TLS 1.3, der Einsatz von Cipher Suites mit Perfect Forward Secrecy, OCSP Stapling, Certificate Transparency sowie HTTP Strict Transport Security diskutiert und im Einklang mit der BSI-Technischen Richtlinie TR-02102-2 bewertet.

Index Terms—Transport Layer Security, TLS Handshake, Downgrade-Angriff, POODLE, Cipher Suite, Perfect Forward Secrecy, Zertifikatsvalidierung, Public Key Infrastructure, Härungsmaßnahmen, IT-Sicherheit

I. EINFÜHRUNG

A. Überblick über TLS

Transport Layer Security (TLS) ist ein kryptografisches Sicherheitsprotokoll zur Absicherung der Datenübertragung in Netzwerken [1]. Es wird eingesetzt, um die Kommunikation zwischen zwei Parteien, Client und Server, vor Abhören, Manipulation und Identitätsfälschung zu schützen. TLS ist heute der Standard zur Absicherung von TCP-basierten Anwendungen und wird unter anderem im Internetprotokoll HTTPS verwendet. TLS ist der Nachfolger von Secure Socket Layer (SSL). Obwohl häufig noch von „SSL“ gesprochen wird, kommen in modernen Systemen ausschließlich TLS-Versionen zum Einsatz. TLS arbeitet zwischen der Anwendungs- und der Transportschicht und ermöglicht eine sichere Kommunikation über unsichere Netzwerke wie das Internet.

B. Ziele von TLS

TLS verfolgt drei wesentliche Sicherheitsziele:

- **Vertraulichkeit (Confidentiality):** Schutz der übertragenen Daten vor unbefugtem Mitlesen durch Verschlüsselung.
- **Integrität (Integrity):** Sicherstellung, dass Daten während der Übertragung nicht verändert oder manipuliert wurden.
- **Authentifizierung (Authentication):** Überprüfung der Identität des Kommunikationspartners mithilfe digitaler Zertifikate.

C. Grundlegende Funktionsweise von TLS

TLS kombiniert verschiedene kryptografische Verfahren:

Symmetrische Verschlüsselung

Die eigentliche Datenübertragung erfolgt mithilfe symmetrischer Verschlüsselung. Dabei verwenden Client und Server denselben geheimen Sitzungsschlüssel (Session Key). Symmetrische Verfahren sind sehr effizient und eignen sich daher besonders für die schnelle Verschlüsselung großer Datenmengen.

Asymmetrische Verschlüsselung

Zur sicheren Aushandlung bzw. zum Schutz der Sitzungsschlüssel werden asymmetrische Verfahren eingesetzt. Hierbei existiert ein Schlüsselpaar aus Public Key und Private Key. In modernen TLS-Versionen erfolgt der Schlüsselaustausch hauptsächlich über ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) [1]. Dadurch kann ein gemeinsamer geheimer Schlüssel erzeugt werden, ohne dass dieser direkt übertragen werden muss.

Integritätsschutz

TLS stellt sicher, dass Daten während der Übertragung nicht manipuliert werden. Ältere TLS-Versionen verwendeten dafür MACs (Message Authentication Codes). Moderne TLS-Versionen, insbesondere TLS 1.3, nutzen überwiegend sogenannte AEAD-Verfahren (Authenticated Encryption with Associated Data), die Verschlüsselung und Integritätsschutz kombinieren [1].

Authentifizierung

Die Authentifizierung erfolgt über digitale Zertifikate. Diese bestätigen die Identität des Servers und ermöglichen dem Client die Überprüfung, ob er tatsächlich mit dem gewünschten Kommunikationspartner verbunden ist.

D. Die TLS-Verbindung

Verbindungsaufbau

Eine TLS-Verbindung besteht im Wesentlichen aus vier Phasen:

- 1) TLS-Handshake
- 2) Authentifizierung und Zertifikatsprüfung
- 3) Schlüsselaustausch und Schlüsselableitung
- 4) Verschlüsselte Datenübertragung

In der Abbildung 1 im Anhang (Abschnitt A) zum TLS-1.3-Handshake ist der grundlegende Ablauf des Verbindungsaufbaus zwischen Client und Server dargestellt. Zunächst sendet der Client eine Client Hello-Nachricht an den Server. Diese enthält unter anderem die unterstützte TLS-Version, eine Liste unterstützter Cipher Suites, Zufallswerte (Client Random) sowie Parameter für den Schlüsselaustausch.

Der Server antwortet anschließend mit einer Server Hello-Nachricht. Darin teilt der Server die ausgewählte Cipher Suite mit und übermittelt sein digitales Zertifikat sowie eine digitale Signatur. Mithilfe des Zertifikats kann sich der Server gegenüber dem Client authentifizieren. Der Client überprüft daraufhin die Gültigkeit des Zertifikats sowie die Signatur der Zertifizierungsstelle.

Nach erfolgreicher Verifikation führen beide Kommunikationspartner den Schlüsselaustausch durch und erzeugen daraus einen gemeinsamen Sitzungsschlüssel (Session Key). Dieser Schlüssel wird anschließend für die symmetrische Verschlüsselung der weiteren Kommunikation verwendet. Nachdem der Handshake abgeschlossen wurde, kann die eigentliche verschlüsselte Datenübertragung beginnen.

Cipher Suites

Eine Cipher Suite beschreibt die kryptografischen Verfahren, die während der TLS-Verbindung verwendet werden. Dazu gehören unter anderem:

- Verfahren zum Schlüsselaustausch
- Verschlüsselungsalgorithmen
- Verfahren zur Integritätsprüfung

Client und Server müssen sich während des Handshakes auf eine gemeinsame Cipher Suite einigen.

Unterschiede zu älteren TLS-Versionen: In älteren TLS-Versionen wurde häufig RSA für den Schlüsselaustausch verwendet. Dabei erzeugte der Client einen geheimen Sitzungsschlüssel und verschlüsselte diesen mit dem öffentlichen Schlüssel des Servers.

TLS 1.3 unterstützt dieses Verfahren nicht mehr. Stattdessen wird überwiegend ECDHE verwendet, da dieses Verfahren sicherer ist und sogenannte Perfect Forward Secrecy ermöglicht. Dadurch bleiben vergangene Kommunikationsdaten selbst dann geschützt, wenn ein langfristiger Schlüssel kompromittiert wird.

Authentifizierung des Servers

Der Server authentifiziert sich gegenüber dem Client mithilfe eines digitalen Zertifikats. Dieses Zertifikat enthält unter anderem:

- den Domainnamen
- den öffentlichen Schlüssel des Servers
- der Gültigkeit
- Informationen über die ausstellende Zertifizierungsstelle
- die digitale Signatur der Certificate Authority (CA)

Während des TLS-Handshakes weist der Server nach, dass er im Besitz des zugehörigen Private Keys ist. Der Client überprüft anschließend die digitale Signatur des Zertifikats und kontrolliert, ob dieses von einer vertrauenswürdigen Zertifizierungsstelle ausgestellt wurde. Dadurch kann der Client sicherstellen, dass der Server authentisch ist.

Zertifikatsprüfung und Vertrauensketten

Die grundlegende Idee eines TLS-Zertifikats besteht darin, einen öffentlichen Schlüssel eindeutig mit einer Identität - meist einem Domainnamen - zu verknüpfen. Der Client vertraut dabei nicht direkt dem Serverzertifikat selbst, sondern der Zertifizierungsstelle, die dieses Zertifikat signiert hat.

Die Vertrauenskette bei TLS sieht typischerweise wie folgt aus:

```
Root CA → signiert → Intermediate CA →  
signiert → Serverzertifikat
```

Die Root Certificate Authority ist selbstsigniert und befindet sich bereits im sogenannten Trust Store des Betriebssystems oder Browsers. Dieser Trust Store enthält vertrauenswürdige Root-Zertifikate.

Aus Sicherheitsgründen signieren Root CAs Serverzertifikate normalerweise nicht direkt. Stattdessen werden sogenannte Intermediate CAs verwendet. Dies bietet mehrere Vorteile:

- höhere Sicherheit
- einfacherer Widerruf kompromittierter Zertifikate
- geringeres Risiko bei einer Kompromittierung der Root CA

Die Intermediate CA signiert anschließend das eigentliche Serverzertifikat.

Überprüfung der Zertifikatskette: Beim Verbindungsaufbau überprüft der Browser bzw. Client mehrere Punkte:

- Ist die digitale Signatur korrekt?
- Ist die Zertifikatskette vollständig?
- Ist die Root CA vertrauenswürdig?
- Ist das Zertifikat noch gültig?
- Passt der Domainname zum Zertifikat?

Kann der Client die Zertifikatskette erfolgreich bis zu einer vertrauenswürdigen Root CA zurückverfolgen, wird das Zertifikat als vertrauenswürdig akzeptiert und die Verbindung hergestellt.

Schlüsselaustausch und Session Keys

Nach erfolgreicher Authentifizierung führen Client und Server einen kryptografischen Schlüsselaustausch durch. In modernen TLS-Versionen erfolgt dies meist über ECDHE. Dabei erzeugen beide Kommunikationspartner gemeinsam einen geheimen Sitzungsschlüssel, ohne diesen direkt zu übertragen. Aus diesem Schlüsselaustausch werden anschließend die Session Keys für die symmetrische Verschlüsselung abgeleitet.

Verschlüsselte Datenübertragung

Nachdem der Handshake abgeschlossen wurde und beide Seiten gemeinsame Sitzungsschlüssel besitzen, beginnt die eigentliche Kommunikation. Die übertragenen Daten werden nun symmetrisch verschlüsselt. Zusätzlich wird ihre Integrität überprüft.

Integritätsschutz mit MACs und AEAD

Bei älteren TLS-Versionen wurde ein Message Authentication Code (MAC) verwendet. Dabei berechnen Sender und Empfänger mithilfe des gemeinsamen geheimen Schlüssels einen Prüfwert über die übertragenen Daten. Stimmen die berechneten Werte überein, wurde die Nachricht nicht manipuliert. Moderne TLS-Versionen verwenden überwiegend AEAD-Verfahren wie AES-GCM oder ChaCha20-Poly1305. Diese kombinieren Verschlüsselung und Integritätsschutz in einem einzigen Verfahren.

E. Zusammenfassung

TLS ermöglicht eine sichere Kommunikation über unsichere Netzwerke. Dazu kombiniert das Protokoll asymmetrische und symmetrische Kryptografie, digitale Zertifikate sowie kryptografische Integritätsprüfungen. Der TLS-Handshake dient dabei der sicheren Aushandlung aller benötigten Parameter und Schlüssel. Zertifikate und Vertrauensketten ermöglichen die Authentifizierung des Servers. Anschließend erfolgt die verschlüsselte und integritätsgeschützte Datenübertragung mithilfe gemeinsamer Sitzungsschlüssel.

II. KONFIGURATIONEN VON TLS

Dieses Kapitel beschreibt ein TestszENARIO, das ein gehärtetes und ein verwundbares TLS-System gegenüberstellt.

A. Sichere Konfiguration

Client-Server-Paar 1 simuliert eine moderne, sichere TLS-Konfiguration. Die wichtigsten Parameter sind Protokollversion, Schlüsselaustausch, Verschlüsselungsmodus, Integritätsschutz, Zertifikatstyp und Schlüssellänge.

Als Protokollversion wird TLS 1.3 verwendet [1]. Der Schlüsselaustausch erfolgt über ECDHE, sodass Perfect Forward Secrecy erreicht wird. Verschlüsselung und Integritätsschutz setzt das Paar über AEAD (z. B. GCM) um [1]. Als Zertifikat dient ein CA-signiertes Zertifikat; Empfehlungen zu Algorithmen und Schlüssellängen siehe BSI-TR [2].

B. Veraltete Konfiguration

Das Client-Server-Paar 2 stellt ein ungepatchtes Altsystem dar und dient als Primärziel für die späteren Fehler-Simulationen. Es beinhaltet die unsichere TLS-Konfiguration.

Es verwendet TLS 1.2 mit rückwärtskompatiblen, unsicheren Mechanismen [3]. Der Schlüsselaustausch erfolgt statisch per RSA (siehe Empfehlungen zu Schlüssellängen) [2], wodurch keine Forward Secrecy besteht. Verschlüsselung per CBC und Integritätsschutz per SHA-1 MAC sind anfällig für Padding-Angriffe. Das Zertifikat ist selbstsigniert und verwendet nur 1024-Bit RSA [2].

C. Konfiguration der Simulationen

Die exakten OpenSSL-Befehle und das Setup sind im Anhang (Abschnitt B) dokumentiert. Dort finden Sie die Zertifikatsgenerierung, Client-Server-Setups sowie die Szenarien für erfolgreichen und nicht-erfolgreichen Downgrade.

III. EXPERIMENTELLES SETUP UND METHODIK

A. Simulationsumgebung

Für die praktische Analyse wurde eine lokale Loopback-Umgebung (127.0.0.1) auf einem Windows-System eingerichtet. OpenSSL 3.6.1 diente auf Client und Server als kryptographische Grundlage [4]. Der Datenverkehr wurde parallel mit Wireshark aufgezeichnet, um den Handshake-Verlauf und Fehlermeldungen auf Paketebene zu analysieren.

B. TLS-Fehlersimulation und Log-Analyse

Es werden ein Downgrade- und ein Handshake-Fehlerfall simuliert, um zu untersuchen, wo Schutzmechanismen greifen oder versagen.

C. Szenario 1: Nicht-erfolgreicher Downgrade (Handshake-Fehlerfall)

Im ersten Experiment wurde untersucht, wie ein moderner Client reagiert, wenn ein Server ein veraltetes, unsicheres Protokoll erzwingt. Dies simuliert das Verhalten bei einer serverseitigen Fehlkonfiguration oder einem Manipulationsversuch im Netzpfad [5].

Vorbereitung der Zertifikate

Zunächst wurde durch die Nutzung von OpenSSL ein selbstsigniertes X.509-Zertifikat mit einem 2048-Bit RSA-Schlüsselpaar generiert. Der Parameter `-nodes` (No-DES) sorgt dafür, dass der private Schlüssel unverschlüsselt bleibt, um einen reibungslosen Serverstart ohne Passwortabfrage zu ermöglichen [4]:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem  
-out cert.pem -days 365 -nodes
```

Verbindungsaufbau

Der simulierte Server wurde so gestartet, dass er ausschließlich das veraltete Protokoll TLS 1.0 anbietet. Anschließend versuchte der reguläre Client, eine sichere Verbindung zu diesem Port aufzubauen. Siehe Anhang C.

Analyse der Handshake-Schritte

Der Verbindungsaufbau wird durch den Client gestartet, worauf der Server antwortet und versucht, TLS 1.0 aufzuzwingen [1]. An dieser Stelle greift der clientseitige Sicherheitsmechanismus: Moderne OpenSSL-Bibliotheken blockieren standardmäßig die Kommunikation über Protokolle unterhalb von TLS 1.2 [4]. Der Handshake bricht unmittelbar ab. Das Terminal des Clients dokumentiert diesen fatalen Protokollkonflikt wie in Anhang D aufgezeigt.

In der parallelen Wireshark-Aufzeichnung zeigt sich dieser Abbruch durch ein rot markiertes Paket mit der Kennzeichnung Alert (Level: Fatal,

Description: Protocol Version (70)). Der Fehlercode 70 (*protocol_version*) belegt, dass der Client die erzwungene Abwärtskompatibilität des Servers zum Schutz des Anwenders blockiert hat [4].

D. Szenario 2: Erfolgreicher Downgrade (Schwächung der Mechanismen)

Das zweite Experiment simuliert den Fall, in dem die clientseitigen Schutzmechanismen bewusst deaktiviert werden – beispielsweise, um die Kommunikation mit ungenügend gehärteten Altsystemen (Legacy-Systemen) im Firmennetzwerk zu erzwingen [5].

Verbindungsaufbau unter reduzierten Sicherheitsbarrieren

Nun werden der Server und der Client durch das explizite Herabsetzen des Sicherheitslevels auf die Stufe Null (@SECLEVEL=0) jeweils gezwungen, ihre Schutzschilde zu deaktivieren [4] (Anhang E).

Analyse der Handshake-Schritte

Durch die Modifikation signalisiert der Client bereits im *Client Hello*, dass er im Notfall auch das unsichere TLS 1.0 akzeptiert [1]. Der Server antwortet im *Server Hello* mit der Bestätigung der veralteten Version. Da der Client das nicht mehr blockiert, wechselt die Verbindung in den Zustand CONNECTED. Anhang F zeigt das Protokoll-Log des Terminals des Clients.

Bewertung der Schwachstelle

Mit abgeschwächten Sicherheitsregeln ist die Verbindung zwar aufgebaut (CONNECTED), aber nicht mehr zeitgemäß geschützt. TLS 1.0 bietet keine ausreichende Vertraulichkeit und Integrität, so dass ein Man-in-the-Middle-Angriff möglich wird [5], [6].

E. Analytische Gegenüberstellung der Simulationsergebnisse

Zur übersichtlichen Strukturierung der Ergebnisse sind die Auswirkungen der beiden Konfigurationen in Tabelle I im Anhang (Abschnitt G) zusammenfassend gegenübergestellt.

IV. AUSWIRKUNGEN EINES FEHLERFALLS AUF SCHUTZZIELE

Die Auswirkungen der beiden untersuchten Szenarien sind in Tabelle II im Anhang (Abschnitt H) zusammengefasst.

Angriff POODLE

Der POODLE-Angriff (Padding Oracle On Downgraded Legacy Encryption, CVE-2014-3566) zeigt, wie ein Angreifer eine Verbindung von TLS auf SSL 3.0 downgraded und dadurch die Verbindung kompromittiert [7]. Die Padding-Oracle-Eigenschaft von SSL 3.0 im CBC-Modus ermöglicht die iterative Rekonstruktion von Klartextbytes durch einen Angreifer im Netzpfad (*Vertraulichkeit*). Manipulierte CBC-Blöcke werden akzeptiert, sofern das Padding zufällig korrekt ist – die MAC-then-Encrypt-Konstruktion bietet keinen ausreichenden Schutz (*Integrität*). Mit dem erzwungenen Wechsel auf SSL 3.0 entfallen die Finished-Message-Signaturen moderner TLS-Versionen; ohne TLS_FALLBACK_SCSV bleibt der Downgrade unbemerkt (*Authentizität*) [8].

Selbstsigniertes Zertifikat

Präsentiert ein Server ein selbstsigniertes Zertifikat, bricht ein korrekt konfigurierter Client den Handshake mit einem CERTIFICATE_UNKNOWN-Alert ab – die Verbindung kommt nicht zustande [6]. Deaktiviert ein Nutzer diese Prüfung jedoch (z. B. durch Akzeptieren einer Browser-Warnung oder mittels --no-verify-Flag), öffnet sich ein Angriffsfenster: Ein Angreifer im Netzpfad kann ein eigenes selbstsigniertes Zertifikat einschleusen, das der Client mangels CA-Prüfung ebenfalls akzeptiert. Damit terminiert der Angreifer *beide* TLS-Verbindungen separat – zum Client und zum echten Server – und leitet Daten transparent weiter (*Man-in-the-Middle*). Alle übertragenen Klartextdaten sind dem Angreifer zugänglich, obwohl der Nutzer eine https://-Verbindung im Browser sieht (*Vertraulichkeit verletzt, Authentizität verletzt*). Die TLS-Record-MAC-Prüfung schützt die Integrität *innerhalb* jeder der beiden Teilverbindungen – der Angreifer kann jedoch Inhalte modifizieren, bevor er sie in der zweiten Verbindung weitersendet, sodass der Endnutzer manipulierte Daten als integer wahrnimmt (*Integrität de facto verletzt*).

V. HÄRTUNGSMASSNAHMEN

TLS Versionsbeschränkung

Das BSI schreibt in der **Technischen Richtlinie TR-02102-2** (Stand: Januar 2026) vor, dass in neuen Systemen ausschließlich TLS 1.2 und TLS 1.3 eingesetzt werden sollen [2]. SSL 2.0, SSL 3.0 und TLS 1.0 sind vollständig zu deaktivieren; TLS 1.1 wird ebenfalls nicht mehr empfohlen. Die Konfiguration in nginx lautet entsprechend:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

Durch die Abschaltung aller Legacy-Protokolle entfällt die Angriffsfläche für POODLE und BEAST [9] grundlegend, da kein Protokoll-Fallback mehr stattfinden kann [8]. Ergänzend sollte **TLS_FALLBACK_SCSV** (RFC 7507) implementiert werden: Übermittelt ein Client diesen Signaling Cipher Suite Value in einem erneuten, herabgestuften Verbindungsversuch, bricht der Server den Handshake mit einem inappropriate_fallback-Alert ab und verhindert so auch aktive Downgrade-Versuche bei Systemen, die Legacy-Versionen aus Kompatibilitätsgründen noch führen [10].

Sichere Cipher Suites

Algorithmen ohne kryptographische Langzeitsicherheit – darunter RC4, DES, 3DES, Export-Cipher und MD5-basierte MACs – sind vollständig aus der Konfiguration zu entfernen [2]. Gemäß BSI TR-02102-2 sind ausschließlich Cipher Suites mit **Perfect Forward Secrecy (PFS)** zu verwenden: Nur ephemere Schlüsselaustauschverfahren (ECDHE, DHE) stellen sicher, dass ein nachträglich kompromittierter Langzeitschlüssel keine bereits aufgezeichneten Sessions entschlüsselbar macht. Empfohlene Suites für TLS 1.2 sind:

```
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
```

Für TLS 1.3 sind die verfügbaren Cipher Suites protokollseitig bereits auf AEAD-Verfahren (AES-GCM, ChaCha20-Poly1305) beschränkt; RC4, CBC sowie der statische RSA-Schlüsseltransport sind im Standard nicht mehr vorgesehen [11], [12].

Zertifikatsvalidierung und Vertrauensketten

Selbstsignierte Zertifikate sind in produktiven Umgebungen unzulässig. Es sind Zertifikate einer anerkannten Zertifizierungsstelle einzusetzen, deren Root-Zertifikate in den Betriebssystem- und Browser-Trust-Stores enthalten sind. Zur weiteren Absicherung der Vertrauenskette empfehlen sich folgende Mechanismen:

- **Certificate Transparency (CT):** Alle ausgestellten Zertifikate werden in öffentliche, append-only-Logs eingetragen. Clients und Monitore können überprüfen, ob ein präsentiertes Zertifikat dort verzeichnet ist, und unaufgeforderte Ausstellungen frühzeitig erkennen.
- **OCSP Stapling:** Der Server liefert den aktuellen Sperrstatus seines Zertifikats unmittelbar im TLS-Handshake mit, sodass der Client keinen separaten OCSP-Request senden muss. Dies verhindert, dass ein Angreifer OCSP-Abfragen blockiert und ein bereits widerrufenes Zertifikat weiterhin nutzbar bleibt.
- **CAA-DNS-Einträge:** *Certification Authority Authorization* Records schränken auf DNS-Ebene ein, welche Zertifizierungsstellen für eine Domain überhaupt Zertifikate ausstellen dürfen, und verringern so das Risiko unberechtigter Ausstellungen.

Regelmäßige Überprüfung und Patch-Management

Da Schwachstellen kontinuierlich neu entdeckt werden (vgl. MITRE CVE-Datenbank), ist eine **regelmäßige Überprüfung der TLS-Konfiguration** zwingend erforderlich. Werkzeuge wie testssl.sh oder der Qualys SSL Labs Server Test erlauben eine automatisierte Analyse aktiver Protokollversionen, Cipher Suites und Zertifikatseigenschaften. Das BSI macht die TR-02102-2 für Bundesbehörden gemäß § 8 Abs. 1 BSIG verbindlich und aktualisiert die Richtlinie fortlaufend – zuletzt im Januar 2026 [13].

ANHANG A TLS-1.3 HANDSHAKE DIAGRAMM

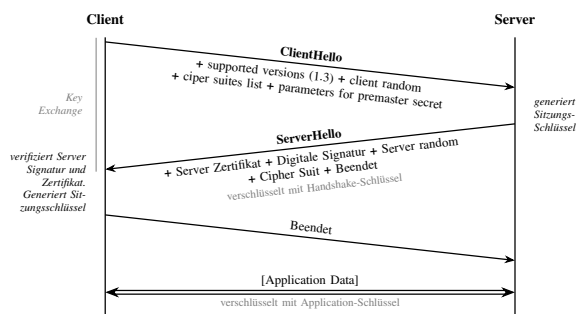


Abbildung 1. TLS 1.3 Handshake

ANHANG B DETAILLIERTE SIMULATIONS-KONFIGURATION

Generierung der Zertifikate

```
# CSP 1: ECDSA P-256 Zertifikat (sicheres Paar)
openssl ecparam -name prime256v1 -genkey -noout \
-out key_secure.pem

openssl req -new -x509 -key key_secure.pem \
-out cert_secure.pem \
-days 365 -subj "/CN=secure.local"

# CSP 2: RSA 1024 Bit, selbstsigniert (unsicheres Paar)
openssl req -x509 -newkey rsa:1024 \
-keyout key_insecure.pem \
-out cert_insecure.pem -days 365 -nodes \
-subj "/CN=insecure.local"
```

CSP 1 – Sicheres Paar

Terminal 1 – Server:

```
openssl s_server -accept 4433 \
-cert cert_secure.pem \
-key key_secure.pem \
-tls1_3
```

Terminal 2 – Client:

```
openssl s_client -connect localhost:4433 \
-tls1_3 \
-CAfile cert_secure.pem
```

Erwartetes Ergebnis:

```
Protocol : TLSv1.3
Cipher   : TLS_AES_256_GCM_SHA384
```

CSP 2 – Unsicheres Paar

Terminal 1 – Server:

```
openssl s_server -accept 4434 \
-cert cert_insecure.pem \
-key key_insecure.pem \
-tls1_2 \
-cipher "AES128-SHA:AES256-SHA@SECLEVEL=0"
```

Terminal 2 – Client:

```
openssl s_client -connect localhost:4434 \
-tls1_2 \
-cipher "AES128-SHA@SECLEVEL=0" \
-no-CAfile
```

Erwartetes Ergebnis:

```
Protocol : TLSv1.2
Cipher   : AES128-SHA
```

Fehler- und Downgrade-Szenarien

Nicht-erfolgreicher Downgrade: Terminal 1 – Server (sicher):

```
openssl s_server -accept 4433 \
-cert cert_secure.pem \
-key key_secure.pem \
-tls1_3
```

Terminal 2 – Client:

```
# Client versucht TLS 1.0 zu erzwingen -- Erwartet:  
alert protocol version (70)  
openssl s_client -connect localhost:4433 \  
-tls1 \  
-cipher "DEFAULT@SECLEVEL=0"
```

Erfolgreicher Downgrade: **Terminal 1 – Server (unsicher):**

```
openssl s_server -accept 4434 \  
-cert cert_insecure.pem \  
-key key_insecure.pem \  
-tls1_2 \  
-cipher "AES128-SHA:AES256-SHA@SECLEVEL=0"
```

Terminal 2 – Client:

```
# Client akzeptiert TLS 1.0 explizit -- Erwartet:  
CONNECTED, Protocol: TLSv1  
openssl s_client -connect localhost:4434 \  
-tls1 \  
-cipher "DEFAULT@SECLEVEL=0" \  
-no-CAfile
```

ANHANG C OPENSSL-BEFEHLE SZENARIO 1

Server-Befehl (TLS 1.0 erzwingen):

```
openssl s_server -accept 4433 \  
-cert cert.pem -key key.pem -tls1
```

Client-Befehl (Standard-Konfiguration):

```
openssl s_client -connect localhost:4433
```

ANHANG D LOG-OUTPUT SZENARIO 1

```
CONNECTED(000001B8)  
FC270000:error:0A00042E:SSL routines:  
ssl3_read_bytes:  
tlsv1 alert protocol version:../openssl-3.6.1/  
ssl/record/  
rec_layer_s3.c:918:SSL alert number 70  
---  
no peer certificate available
```

ANHANG E OPENSSL-BEFEHLE SZENARIO 2

Server-Befehl:

```
openssl s_server -accept 4433 \  
-cert cert.pem -key key.pem -tls1 \  
-cipher "DEFAULT@SECLEVEL=0"
```

Client-Befehl:

```
openssl s_client -connect localhost:4433 -tls1 \  
-cipher "DEFAULT@SECLEVEL=0"
```

ANHANG F LOG-OUTPUT SZENARIO 2

```
CONNECTED(000001BC)  
Can't use SSL_get_servername  
depth=0 C=AU, ST=Some-State  
verify error:num=18:self-signed certificate  
verify return:1  
---  
Certificate chain  
0 s:C=AU, ST=Some-State  
i:C=AU, ST=Some-State  
a:PKEY: RSA, 2048 (bit); sigalg:  
sha256WithRSAEncryption  
---  
New, TLSv1.0, Cipher is ECDHE-RSA-AES256-SHA  
Protocol : TLSv1  
Server public key is 2048 bit  
Secure Renegotiation IS NOT supported
```

ANHANG G VERGLEICHSTABELLE DER SIMULATIONEN

Tabelle I
VERGLEICHSMATRIX DER SIMULIERTEN TLS-FEHLERFÄLLE

Kriterium	Szenario 1: Mismatch	Szenario 2: Downgrade
Server-Vorgabe	Erzwingung TLS 1.0	Erzwingung TLS 1.0
Client-Konfiguration	Standard (Level ≥ 1)	Deaktiviert (Level 0)
Handshake-Verlauf	Abbruch (Fatal Alert)	Erfolgreich (CONNECTED)
Log-Indikator	SSL alert number 70	Protocol: TLSv1
Sicherheitszustand	Sicher (Leitung blockiert)	Insecure (vulnerabel)

ANHANG H TABELLE DER AUSWIRKUNGEN AUF SCHUTZZIELE

Tabelle II
AUSWIRKUNGEN AUF DIE SCHUTZZIELE (CIA)

Schutzziel	Angriff POODLE [7]	Self-Signed-Cert
Vertraulichkeit	✗ Verletzt	○ Gefährdet
Integrität	✗ Verletzt	○ Partiiell
Authentizität	✗ Verletzt	✗ Verletzt

✗ = verletzt; ○ = partiell gefährdet

LITERATUR

- [1] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF, RFC 8446, Aug. 2018, [Online]. Verfügbar: <https://www.rfc-editor.org/info/rfc8446>; Zugriff: 10. Mai 2026.
- [2] Bundesamt für Sicherheit in der Informationstechnik (BSI), "Technische Richtlinie TR-02102-2: Kryptographische Verfahren – Verwendung von Transport Layer Security (TLS)," BSI, Tech. Rep., 2026, version 2026-01. [Online]. Verfügbar: <https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf>; Zugriff: 07. Mai 2026.
- [3] K. Moriarty and S. Farrell, "Deprecating TLS 1.0 and TLS 1.1," IETF, RFC 8996, Mar. 2021, [Online]. Verfügbar: <https://www.rfc-editor.org/info/rfc8996>; Zugriff: 13. Mai 2026.
- [4] OpenSSL Project, "s_client / s_server Manual Pages," OpenSSL Project. [Online]. Verfügbar: <https://docs.openssl.org/>; zugriff: 13. Mai 2026.

- [5] SentinelOne, “Downgrade-Angriffe: Arten, Beispiele und Prävention,” SentinelOne Cyber Security 101. [Online]. Verfügbar: <https://www.sentinelone.com/de/cybersecurity-101/cybersecurity/downgrade-attacks/>, zugriff: 02. Mai 2026.
- [6] MDN Web Docs, “Transport Layer Security (TLS)-Konfiguration,” [Online]. Verfügbar: <https://developer.mozilla.org/>, zugriff: 02. Mai 2026.
- [7] NIST National Vulnerability Database, “CVE-2014-3566: SSL 3.0 Padding Oracle Vulnerability,” [Online]. Verfügbar: <https://nvd.nist.gov/vuln/detail/CVE-2014-3566>, 2014, zugriff: 07. Mai 2026.
- [8] Thomas-Krenn-Wiki, “SSL 3.0 POODLE Attack,” [Online]. Verfügbar: https://www.thomas-krenn.com/de/wiki/SSL_3.0_POODLE_Attack, 2023, zugriff: 05. Mai 2026.
- [9] NIST National Vulnerability Database, “CVE-2011-3389: TLS 1.0 CBC Predictable IV (BEAST),” [Online]. Verfügbar: <https://nvd.nist.gov/vuln/detail/CVE-2011-3389>, 2011, zugriff: 10. Mai 2026.
- [10] J. Hoffman and M. Kucherawy, “Transport layer security (TLS) Fallback Signaling Cipher Suite Value (SCSV),” IETF, RFC 7507, Apr. 2015, [Online]. Verfügbar: <https://www.rfc-editor.org/info/rfc7507>; Zugriff: 05. Mai 2026.
- [11] Der Windows Papst, “Recommended Cipher Suites 2025,” [Online]. Verfügbar: <https://www.der-windows-papst.de/2024/06/24/recommended-cipher-suites-2025/>, Jun. 2024, zugriff: 13. Mai 2026.
- [12] Mozilla Foundation, “SSL Configuration Generator,” [Online]. Verfügbar: <https://ssl-config.mozilla.org/>, zugriff: 05. Mai 2026.
- [13] kes – Zeitschrift für Informations-Sicherheit, “Technische Richtlinien des BSI zur Kryptografie,” [Online]. Verfügbar: <https://www.kes-informationssicherheit.de>, 2025, zugriff: 07. Mai 2026.